B. P. PODDAR INSTITUTE OF MANAGEMENT AND TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

PPT ON AI PUZZLE SOLVER

Subject	Artificial Intelligence
Sub.Code	PEC-IT501B
Stream	IT
Section	A
Semester	5th
Group	07
Session	2026-27

Name :	Roll No :
Mikki Kumari	11500224006
Sachin Kumar	11500224019
Anandi Kumari	11500224021
Rahul Kumar Yadav	11500224022

Slide-02

Problem Statement

- **Implement a small state-based puzzle solver. Define the initial state, goal state, legal actions and search strategy.**

The 8-Puzzle contains eight numbered tiles and one blank cell on a 3×3 board.

Move one adjacent tile into the blank cell at a time.

Find a legal sequence that transforms the initial arrangement into the goal arrangement.

Objective: find a shortest solution path automatically.

Slide-03

Project Objectives

- Represent the puzzle as a state space.
- Define initial state, goal state, legal actions, and transition rules.
- Implement A* Search with a heuristic function.
- Validate input and detect unsolvable configurations.
- Visualize the solution and report search performance.

Slide-04

AI Technique: A* Search

➡ A* is an informed best-first search algorithm.

- It checks both how far you have already come and how far you still have to go, and then picks the road that looks cheapest in total.

Evaluation function: f(n) = g(n) + h(n).

- f(n) = Estimated total cost.

- g(n): moves already taken; h(n): estimated remaining moves.

Heuristic: Manhattan Distance; ignore the blank tile.

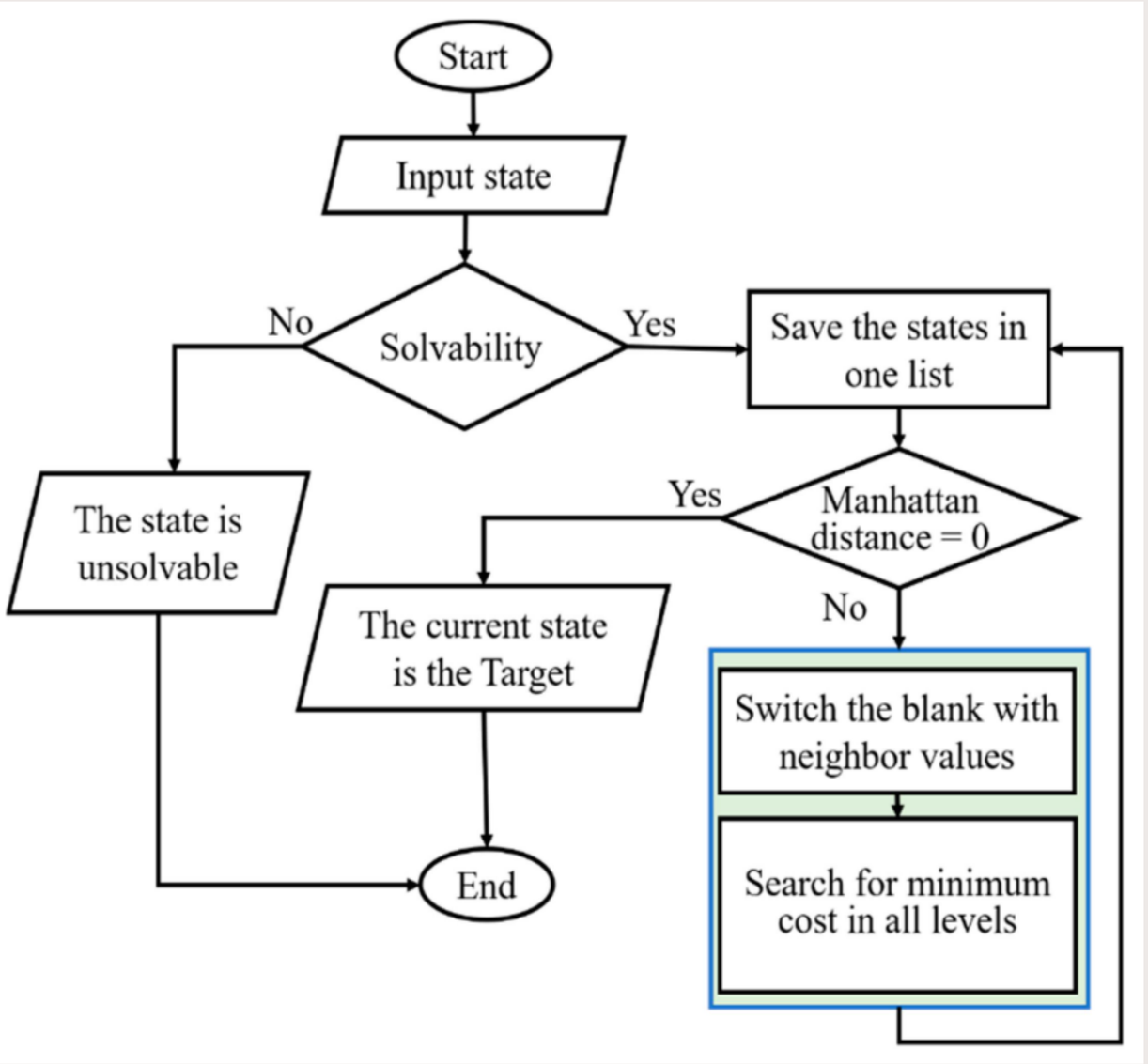
- With an admissible heuristic and unit move costs, A* returns an optimal path.

Manhattan distance = Σ (|current row − goal row| + |current column − goal column|).

Slide-05

System Architecture

- User enters a 9-value initial configuration.
- Validate digits 0–8 and check inversion parity.
- Generate legal successor states.
- A* priority queue selects the state with the smallest f-score.
- Reconstruct the path and display it in the Tkinter interface.



Slide-06

Algorithm/State Model

- **State:** tuple of nine integers; 0 represents the blank.
- **Initial State:**

1	2	3
4	0	6
7	5	8

Goal State:

1	2	3
4	5	6
7	8	0

- **Legal Actions:** Blank Up, Down, Left, Right.
- **Step cost:** 1 per move.
- Parent links reconstruct the ordered solution path.

A* Algorithm Steps:-

i) Start with the Initial State.

ii) Calculate Heuristic Distance.

iii) Select the State with Minimum f(n).

iv) Generate legal neighbouring states.

v) Repeat until the goal state is reached.

vi) Display the Solution Path.

A* pseudocode: pop minimum f; if goal, reconstruct; otherwise expand legal neighbours and update better g-scores.

Slide-07

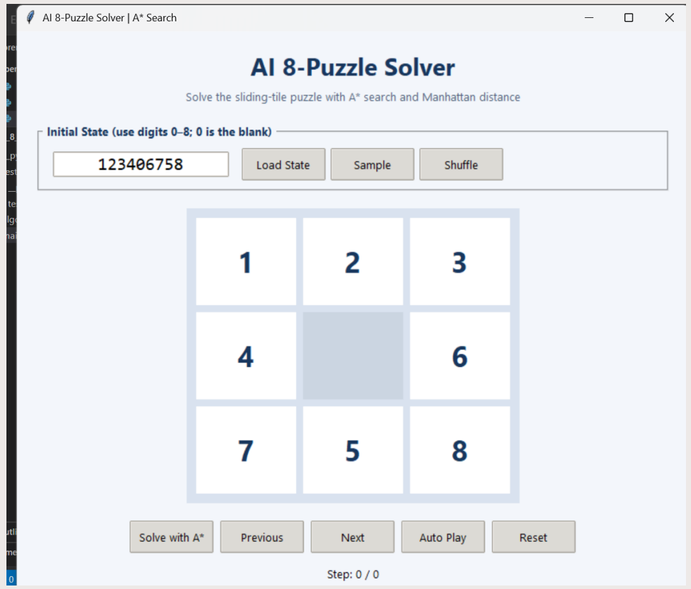
Implementation

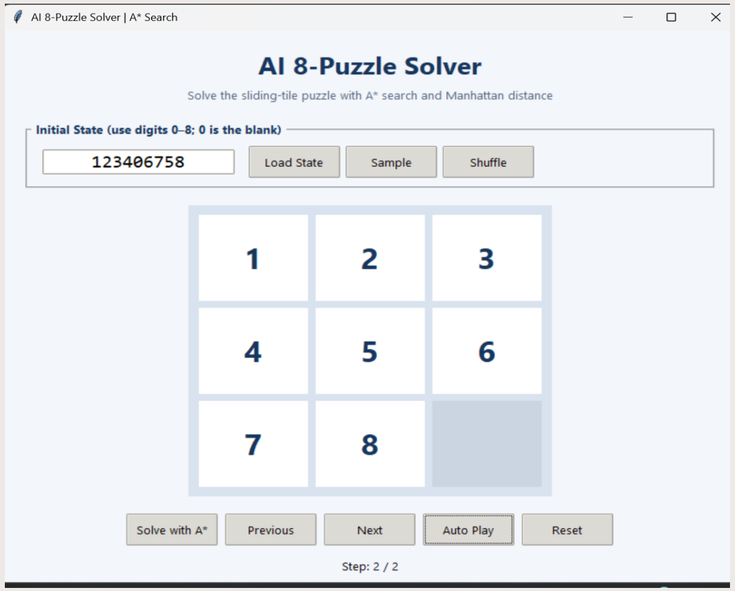
Component	Technology
Language	Python
GUI	Tkinter
Algorithm	A* Search
Heuristic	Manhattan Distance
Data structure	Priority Queue,Dictonaries
IDE	VS Code

- main.py – Graphical interface.
- algorithm.py – A* algorithm and puzzle logic.

Slide-08

Results





Sample initial state: 1 2 3 / 4 0 6 / 7 5 8.

Expected optimal moves: blank Down, then blank Right.

Expected solution cost: 2 moves.

GUI displays expanded states, generated states, and execution time.

Insert genuine application screenshots on this slide before submission.

Slide-09

Conclusions

The project models a classic AI state-space search problem.

A* with Manhattan Distance finds an optimal path for unit-cost moves.

The GUI makes the search result easy to inspect and demonstrate.

Input validation and parity checking improve reliability.

Slide-10

Future Scope

- Add alternative heuristics such as Linear Conflict.
- Support for larger puzzles such as 15-Puzzle.
- Compare A*, BFS, Greedy Best-First, and IDA*.
- Support configurable goals or larger puzzle sizes.
- Add charts and exportable performance reports.
- Improve the interface and add keyboard controls.